

File Handling

Background

- Console oriented I/O functions use keyboard as input device and monitor as output device.
- The I/O functions like printf(), scanf(), getchar(), putchar(), gets(), puts()
- **Problem:**
 1. Entire data is lost when either the program is terminated or the computer is turned off.
 2. When the volume of data to be entered is large, it takes a lot of time to enter the data.
 3. If user makes a mistake while entering data, whole data has to be re-entered.
- **Solution: File**

Concept of file

- ***File***

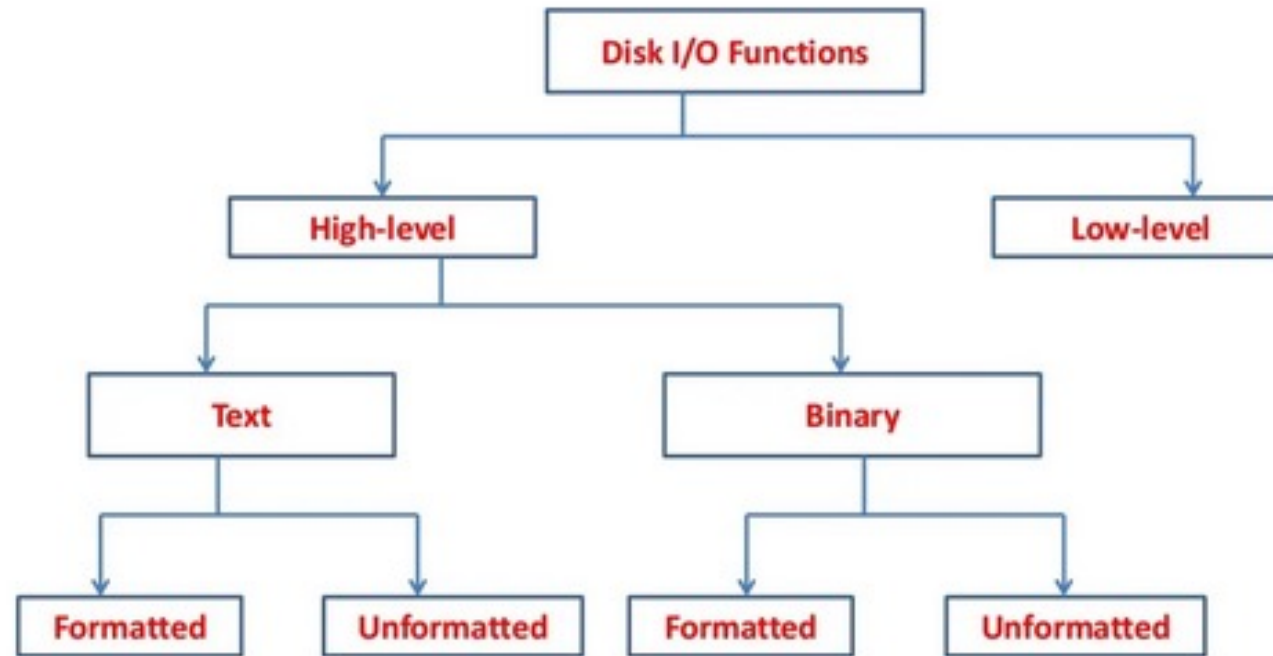
is a place on the *disk* (not memory) where a group of related data is stored. Also called *data files*.

- **The *Data File***

allows us to store information permanently and to access and alter that information whenever necessary.



Classification of disk/file I/O functions



Some high-level I/O functions

Function name	Operation
<code>fopen()</code>	•Creates a new file for use •Opens an existing file for use
<code>fclose()</code>	•Closes a file which was opened for use
<code>fgetc()</code>	•Reads a character from a file
<code>fputc()</code>	•Writes a character to a file
<code>fprintf()</code>	•Writes a set of data values to a file
<code>fscanf()</code>	•Reads a set of data values from a file
<code>fseek()</code>	•Sets the position to a desired point in the file
<code>ftell()</code>	•Gives the current position in the file (in terms of bytes from the start)
<code>rewind()</code>	•Sets the position to the beginning of the file

Defining and Opening a *file*

- The general format for declaring and opening a file is:

FILE *fp;

fp=fopen("filename", "mode");

- Here, the first statement declares the variable *fp* as a "pointer to the data type FILE".
- The second statement opens the file named *filename* with the purpose *mode* and the beginning address of the buffer area allocated for the file is stored by file pointer *fp*.
- *Note: Any no. of files can be opened and used at a time.*



File Opening Modes

- There are mainly six modes:
 1. “r” (i.e. read mode)
 2. “w” (i.e. write mode)
 3. “a” (i.e. append mode)
 4. “r+” (i.e. read and write mode)
 5. “w+” (i.e. write and read mode)
 6. “a+” (i.e. append and read mode)

Closing a *file*

- The closing a file ensures that all outstanding information associated with the file is **flushed out from the buffers and all links to the file are broken**.
- In cases where there is a limit to the no. of files that can be kept open simultaneously, closing of unwanted files help in opening the required ones.
- Another instance where we have to close a file is when we want to reopen the same file in different mode.
- The file is closed using library function *fclose()* as:

fclose(fp);

Library Functions for Reading/Writing from/to a File: File I/O Functions

- Once a file is opened, reading out of or writing to it is accomplished using the standard I/O functions.

String Input/Output Functions

- Using string I/O functions *fgets()* and *fputs()*, data can be read from a file or written to a file in the form of array of characters.
 - i. *fgets()*: is used to read string from file.
Syntax: *fgets(string, int_value, fp);*
Here, *int_value* denotes the no. of characters in the string.
 - ii. *fputs()*: is used to write string to file.
Syntax: *fputs(string, fp);*

/* Program to create a file named test.txt and write some text */

```
#include<conio.h>
#include<stdio.h>
#include<stdlib.h>
void main()
{
FILE *fp;
clrscr();
fp=fopen("D:\\test.txt", "w");
if(fp==NULL)
{
    printf("\n Cannot create file.");
    exit(0);
}
else
{
    printf("\n File is created.");
}
fputs("I study CSIT", fp);

fclose(fp);
getch();
}
```

***/* Program to open the file named test.txt, read its content
and display it to screen */***

```
#include<conio.h>
#include<stdio.h>
#include<stdlib.h>
void main()
{
FILE *fp;
char s[100];
clrscr();
fp=fopen("D:test.txt", "r");
if(fp==NULL)
{
printf("\n Cannot open file.");
exit(0);
}
else
{
printf("\nFile is opened.");
}
fgets(s,19,fp);

printf("\n Text from file is: %s", s);

fclose(fp);
getch();
}
```


**/* Program to open the file named test.txt and add to it the
text "in CAB" */**

```
#include<conio.h>
#include<stdio.h>
#include<stdlib.h>
void main()
{
FILE *fp;
clrscr();
fp=fopen("D:\\test.txt", "a");
if(fp==NULL)
{
printf("\n Cannot open file.");
exit(1);
}
else
{
printf("\n File is opened.");
}
fputs("in CAB", fp);
fclose(fp);
getch();
}
```


Naming a file

```
void main()
{
FILE *fp;
char filename[20];
clrscr();
printf("Enter filename:\t");
gets(filename);
fp=fopen(filename, "w");
if(fp==NULL)
{
printf("\n Cannot create file.");
exit(1);
}
else
{
printf("\n File is created.");
}
getch();
}
```

// If only filename is given, file is created in C:\TC\BIN otherwise file is created in the given path.

Character I/O Functions

- Using character I/O functions *fgetc()* and *fputc()*, data can be read from file or written onto file one character at a time.

i. *fgetc()*: is used to read a character from a file.

Syntax:

char_variable=fgetc(fp);

ii. *fputc()*: is used to write a character to a file.

Syntax:

fputc('character' or character_variable, fp);

/* Program to create a file and write some text to it one character at a time using fputc() function until user hits the enter key*/

```
void main()
{
FILE *fp;
char filename[20];
char c;
clrscr();
printf("Enter filename:\t");
gets(filename);
fp=fopen(filename,"w");
if(fp==NULL)
{
    printf("\n Cannot create file.");
    exit();
}
else
    printf("\n File is created.");
printf("\n Enter your text until Enter key:\n");
while((c=getchar())!='\n')
    fputc(c,fp);
fclose(fp);
getch();
}
```

***/* Program to open a file, read its content one character at a time using
fgetc() function and display it to screen*/***

```
void main()  
{  
FILE *fp;  
char filename[20];  
char c;  
clrscr();  
printf("Enter filename:\t");  
gets(filename);  
fp=fopen(filename, "r");  
if(fp==NULL)  
{  
    printf("\n Cannot open file.");  
    exit();  
}  
  
printf("\n The content of file is:\n");  
while((c=fgetc(fp))!=EOF)  
    putchar(c);  
fclose(fp);  
getch();  
}
```

//EOF=>end-of-file

End-Of-File (EOF)

- EOF is a special character (an integer with ASCII value 26) that indicates that the end-of-file has been reached. This character can be generated from the keyboard by typing **Ctrl+Z**.
- Defined in `<stdio.h>`
- When we are creating a file, the special character EOF, is inserted after the last character of the file by the **Operating System**.
- *Caution: An attempt to read after EOF might either cause the program to terminate with an error or result in an infinite loop situation.*

/* Program to append some text to a file by reading filename from user*/

```
void main()  
{  
FILE *fp;  
char filename[20];  
char c;  
clrscr();  
printf("Enter filename:\t");  
gets(filename);  
fp=fopen(filename,"a");  
if(fp==NULL)  
{  
    printf("\n Cannot create or open file.");  
    exit();  
}  
printf("\nEnter text to append to file %s:\n", filename);  
while((c=getchar())!='\n')  
    fputc(c,fp);  
  
fclose(fp);  
getch();  
}
```


/* Program to open a file and copy all its content to another file*/

```
void main()
{
FILE *sfp,*dfp;
char sfilename[20],dfilename[20];
char c;
clrscr();
printf("Enter source filename:\t");
gets(sfilename);
printf("\n Enter destination
filename:\t");
gets(dfilename);
sfp=fopen(sfilename,"r");
if(sfp==NULL)
{
printf("\nSource file can't be
opened.");
exit();
}
dfp=fopen(dfilename, "w");
if(dfp==NULL)
{
printf("\n Destination file cannot
be created or opened.");
exit();
}
while((c=fgetc(sfp))!=EOF)
fputc(c, dfp);
printf("\n Copied.....");
fclose(dfp);
fclose(sfp);
getch();
}
```

Question

- **Given a text file, create another text file deleting all the vowels (a, e, i, o, u).**


```
void main()
{
FILE *fp,*fpp;
char c;
fp=fopen("C:\\test.txt","r+");
clrscr();
if(fp==NULL)
{
printf("Cannot open file");
exit();
}
fpp=fopen("C:\\hello.txt","w");
if(fpp==NULL)
{
printf("Cannot create file");
exit();
}
```

```
}

while((c=fgetc(fp))!=EOF)
{

if((c!='a')&&(c!='e')&&(c!='i')&&(
c!='o')&&(c!='u'))
fputc(c, fpp);

}

fclose(fp);
fclose(fpp);
getch();
}
```

Formatted I/O Functions

- Using formatted I/O functions, *fprintf()* and *fscanf()*, numbers, characters or string can be read from file or written onto file according to our requirement format.
 - i. *fprintf()*: is formatted output function which is used to write integer, float, char or string value to a file.
Syntax:
fprintf(fp, "control_string", list_of_variables);
 - ii. *fscanf()*: is formatted input function which is used to read integer, float, char or string value from a file.
Syntax:
fscanf(fp, "control_string", &list_of_variables);

/* Program to create a file named student.txt and write name, roll, address and marks of a student to this file*/

```
void main()
{
FILE *fp;
char name[20];
int roll;
char address[20];
float marks;
clrscr();
fp=fopen("C:\\student.txt", "w");
if(fp==NULL)
{
printf("\n File cannot be created or opened.");
exit();
}
```

```
printf("\n Enter name of student:\t");
gets(name);
printf("\n Enter roll number of %s:\t", name);
scanf("%d", &roll);
fflush(stdin);
printf("\n Enter address of %s:\t", name);
gets(address);
printf("\n Enter marks of %s:\t", name);
scanf("%f", &marks);
printf("\n Now writing data to file...");
fprintf(fp, "Name=%s\n Roll=%d\n Address=%s\n Marks=%.2f",
name, roll, address, marks);
printf("\n Completed");
fclose(fp);
getch();
}
```

Use of fflush()

- Here, after providing name of student, we would hit enter key.....**No Problem**.....and then we provide roll of student.....and hit the enter key again.....**Problem**.....
- At this time the enter key which is in the keyboard buffer is read by the *gets()/scanf()* function for address (as enter key is a character, \n), so that we are able to fill only the marks.
- To avoid this problem, we use the function *fflush()*.
- It is designed to remove or flush out any data remaining in the buffer.

Question

- **Define a structure for Vehicle Owner having data members name, address, telephone number, vehicle number and license number. Take the data for ten owners, write them in file “Own.txt”. Read the data from the file and display them.**


```
struct vehicle_owner
```

```
{  
    char name[20];  
    char address[20];  
    long int phone_no;  
    int vehicle_no;  
    int license_no;  
};
```

```
void main()
```

```
{  
    FILE *fp;  
    struct vehicle_owner vehicle[10], v[10];  
    int i;  
    clrscr();  
    fp=fopen("C:\\Own.txt","w");  
    if(fp==NULL)  
    {  
        printf("\nCannot create file.");  
        exit();  
    }
```

```
for(i=0;i<SIZE;i++)
```

```
{  
    printf("\n Enter information about vehicle owner %d",i+1);  
    printf("\n Enter name :t");  
    gets(vehicle[i].name);
```

```
  
    printf("\n Enter address:t");  
    gets(vehicle[i].address);
```

```
  
    printf("\n Enter telephone no:t");  
    scanf("%ld", &vehicle[i].phone_no);
```

```
  
    printf("\n Enter vehicle no:t");  
    scanf("%d", &vehicle[i].vehicle_no);
```

```
  
    printf("\n Enter license no:t");  
    scanf("%d", &vehicle[i].license_no);
```

```
  
    fprintf(fp," %s\t %s\t %ld\t %d\t %d\n", vehicle[i].name, vehicle[i].address,  
    vehicle[i].phone_no, vehicle[i].vehicle_no, vehicle[i].license_no);  
    fflush(stdin);  
}
```

```

fclose(fp);
fp=fopen("C:\\Own.txt","r");
for(i=0;i<10;i++)
{
fscanf(fp,"%s %s %ld %d
      %d",&v[i].name,&v[i].address,&v[i].phone_no,&v[i].vehicle_no,&v[i].license_
no);
printf("%s\t%s\t%ld\t%d\t%d\n",v[i].name,v[i].address,v[i].phone_no,v[i].vehicl
e_no,v[i].license_no);
}
fclose(fp);
getch();
}

```

Problem

- **Given a text file, create another text file deleting the following words “three”, “bad”, and “time”.**


```
#include <stdio.h>
void main()
{
FILE *fp,*fpp;
char c[10];
fp=fopen("C:\\test.txt","r");
clrscr();
if(fp==NULL)
{
printf("Cannot open file");
exit();
}
fpp=fopen("C:\\hello.txt","w");
if(fpp==NULL)
{
printf("Cannot create file");
exit();
}
while(fscanf(fp,"%s",c)!=EOF)
{
if((strcmp(c,"three")!=0)&&(strcmp(c,"bad")!=0)&&(strcmp(c,"time")!=0))
{
fprintf(fpp,"%s ",c);
}
}
fclose(fp);
fclose(fpp);
getch();
}
```

Problem

- **Some text file is given, create another text file replacing the following words “Ram” to “Hari”, “Sita” to “Gita”, and “Govinda” to “Shiva”.**

```
#include <stdio.h>
void main()
{
FILE *fp,*fpp;
char c[10];
fp=fopen("C:\\test.txt","r");
clrscr();
if(fp==NULL)
{
printf("Cannot open file");
exit();
}
fpp=fopen("C:\\hello.txt","w");
if(fpp==NULL)
{
printf("Cannot create file");
exit();
}
```

```
while(fscanf(fp,"%s",c)!=EOF)
{
if(strcmp(c,"Ram")==0)
fprintf(fpp,"Hari ",c);

else if(strcmp(c,"Sita")==0)
fprintf(fpp,"Gita",c);

else if(strcmp(c,"Govinda")==0)
fprintf(fpp,"Shiva",c);
else
fprintf(fpp,"%s ",c);
}
fclose(fp);
fclose(fpp);
getch();
}
```

Question

- **Create a program to create a data file and write the integers from 1 to 20 to this file and then read the numbers from the file to display the squares of the stored numbers.**

```
#include <stdio.h>
void main()
{
    FILE *fp;
    register unsigned int i;
    unsigned filedata;
    clrscr();
    fp=fopen("C:\\data.txt","w");
    if(fp==NULL)
    {
        printf("\nCannot create data file.");
        exit();
    }
    for(i=1;i<21;i++)
    {
        fprintf(fp,"%u\t",i);
    }
    fclose(fp);
    fp=fopen("C:\\data.txt","r");
    printf("\nThe squares of the stored numbers are:\t");
    for(i=1;i<21;i++)
    {
        fscanf(fp,"%u",&filedata);
        filedata=filedata*filedata;
        printf("%u\t",filedata);
    }
    getch();
}
```

Question

- **A file named DATA contains a series of integer numbers. Code a program to read these numbers and then write all odd numbers to a file to be called ODD and all even numbers to a file to be called EVEN.**

```
#include <stdio.h>
void main()
{
FILE *fpdata;
FILE *fpodd;
FILE *fpeven;
int i,n;
int num;
clrscr();
printf("\nHow many integers you want in data file?:\t");
scanf("%d",&n);
printf("\nEnter %d integers:\t",n);
fpdata=fopen("C:\\DATA.txt","w");
for(i=0;i<n;i++)
{
scanf("%d",&num);
fprintf(fpdata,"%d\n",num);
}
fclose(fpdata);
fpdata=fopen("C:\\DATA.txt","r");
fpodd=fopen("C:\\ODD.txt","w");
fpeven=fopen("C:\\EVEN.txt","w");
```

```
for(i=0;i<n;i++)
{
fscanf(fpdata,"%d", &num);

if(num%2==0)
fprintf(fpeven,"%d\t", num);
else
fprintf(fpodd,"%d\t", num);
}

fclose(fpdata);
fclose(fpodd);
fclose(fpeven);
getch();
}
```